

Databases, Schemas & Tables

Module 3: Databases, Schemas & Tables

Purpose: Understand how Yellowbrick organizes data and how to create and manage databases, schemas, and tables confidently.

Main Topics: Object hierarchy · Databases · Schemas · Tables · Data types · Distribution · Ordering · Constraints · Best practices

Audience: Business users, analysts, and anyone creating or managing data objects in Yellowbrick

The Yellowbrick Object Hierarchy

Everything in Yellowbrick lives in a three-level namespace: database . schema . table

DATABASE Top-level container

SCHEMA Logical namespace inside a database. Groups related tables.

TABLE The core storage object. Columns, data types, distribution.

COLUMN Typed data stored in columnar blocks across compute nodes.

Full object reference: `SELECT segment, age_range FROM melco_analytics.marketing.customer_segments;`

What is a Database?

A database is the top-level container in Yellowbrick.

- Top Level container
- An important feature of a database is its encoding
 - Latin, UTF8 - UTF8 is almost always right choice
- Each database has its own schemas, tables and permissions
- As a business user, you will mostly work within an assigned database
- Creating a new database is typically a DBA/admin task

Creating a Database

Syntax

```
CREATE DATABASE database_name
[ WITH ]
[ OWNER = username ]
[ ENCODING = encoding ];
```

Common Examples

```
-- Create a simple database
CREATE DATABASE melco_analytics;

-- Create with a specific owner
CREATE DATABASE melco_analytics
  WITH OWNER = melco_admin;

-- View existing databases
SELECT name, owner_id
FROM   sys.database
ORDER BY name;
```

Key Points

1 Admin User required

Only database Admin users can CREATE DATABASE. As a business user, request new databases through your DBA.

2 Names are case-insensitive (unless quoted)

Yellowbrick stores identifiers in lowercase. Use snake_case (underscores, no spaces) for all object names. [Link](#)

3 Encoding cannot be changed once created

Use UTF8 for almost everything (required for JSON), LATIN9 for legacy.

What is a Schema?

A schema is a named namespace inside a database that groups related tables, views, and other objects.

Why schemas matter

- Organise tables into logical groups (e.g., finance, marketing, staging)
- Control access — grant SELECT on a whole schema, not table-by-table
- Allow the same table name in different schemas with no conflict
- Every database has a public schema by default — all database users can access it
- search_path determines which schema Yellowbrick looks in first when you omit the schema prefix

How search_path works

```
-- Set the schema search order
SET search_path = marketing, public;

-- This now resolves to marketing.customers
SELECT * FROM customers;

-- Or qualify fully to be explicit
SELECT * FROM finance.budget_2024;
```

Tip

Always set your search_path at the start of a session or in your connection string — it avoids having to type the schema prefix on every query and prevents accidentally creating objects in the wrong schema.

Creating a Schema & Setting Your Search Path

Create & Inspect

```
-- Basic schema creation
CREATE SCHEMA marketing;

-- Create with owner
CREATE SCHEMA marketing
    AUTHORIZATION melco_admin;

-- List schemas in current db
SELECT schema_name, schema_owner
FROM    information_schema.schemata
ORDER BY schema_name;

-- List tables in a schema
SELECT table_name
FROM    information_schema.tables
WHERE   table_schema = 'marketing'
ORDER BY table_name;
```

Set Search Path

```
-- For the current session
SET search_path = marketing, public;

-- Permanently for your user
ALTER USER your_username
    SET search_path = marketing, public;

-- Check the current setting
SHOW search_path;
```

Schema Design Patterns

How you organise schemas shapes how teams collaborate and how permissions work.

raw.*

Raw / Ingest

Tables land here directly from source systems — no transformation. Typically write-once; ETL pipelines own this schema.

```
raw.casino_transactions_2024
raw.hotel_bookings_json
```

staging.*

Staging / Work

Intermediate scratch space for cleaning, joining, and transforming data. Temp tables also live here. Refreshed often.

```
staging.cleaned_transactions
staging.daily_summary_work
```

mart.*

Data Mart

Final, ready-to-query tables used by analysts and BI tools. Well-documented and stable — no surprise schema changes.

```
mart.revenue_by_property
mart.vip_customer_segments
```

msuarez.*

Personal Schema

Your own scratch space. Create, drop, and experiment freely without affecting shared work. Named after your user.

```
msuarez.analysis_q1
msuarez.test_join_results
```


What is a Table in Yellowbrick?

Tables are where data lives. In Yellowbrick, every table is a columnar, distributed object — its data is split across compute nodes and stored column-by-column on disk.

- No row-level indexes — range-restricted block scanning does the filtering work
- Primary key & foreign key constraints exist but are NOT enforced — data quality is ensured upstream
- NOT NULL is the only enforced constraint
- Distribution determines which compute node holds each row
- Table design choices (distribution, ordering, data types) directly affect query speed

Data is stored by column, not by row

cust_id	region	spend	date
1001	Macau	4 250	2024-01
1002	Manila	8 100	2024-01
1003	Macau	1 750	2024-02
1004	Manila	6 300	2024-02
block	block	block	block

Each column stored separately on disk → only read what you query

Creating a Table: Basic Syntax

```
CREATE TABLE schema_name.table_name (  
    column_name    data_type    [NOT NULL],  
    column_name    data_type    [NOT NULL],  
    ...  
)  
DISTRIBUTE ON (column_name)    -- or REPLICATE / RANDOM  
[ SORT ON (column_name) ]
```

Qualify with schema.table — Good to be explicit

Column name first, then data type, then constraints

DISTRIBUTE ON — An important performance decision

SORT ON — tells the engine how to order blocks on disk

Real-world example

```
CREATE TABLE mart.casino_revenue (  
    transaction_id    BIGINT        NOT NULL,  
    property_code     VARCHAR(10)   NOT NULL,  
    game_type         VARCHAR(50),  
    revenue_amt       NUMERIC(15,2) NOT NULL,  
    txn_date          DATE          NOT NULL  
)  
DISTRIBUTE ON (transaction_id)  
SORT ON (txn_date);
```

Other useful table creation patterns:

- CREATE TEMP TABLE — session-scoped, auto-drops
- CREATE TABLE AS SELECT — copy data from a query
- CREATE TABLE IF NOT EXISTS — safe re-runs

Choosing the Right Data Type

The right data type compresses better, uses less I/O, and catches bad data early. [Data Type Docs](#)

Category	Type	Use when...	Avoid when...
Integer	INT / INTEGER	Whole numbers up to ~2.1 billion (IDs, counts, codes)	Numbers could exceed 2.1B → use BIGINT
Integer	BIGINT	Large whole numbers, transaction IDs, event counters	Small lookups — wastes storage vs INT
Decimal	NUMERIC(p, s)	Exact decimals: money, rates, measurements. e.g. NUMERIC(15,2)	very wide precision is slow
DOUBLE	DOUBLE	Performance is important, approximations are OK	Accuracy is important.
Text	VARCHAR(n)	Variable-length strings. Set n to realistic max, not MAX	Fixed-length data (e.g. 2-char codes) — CHAR(n) compresses better
Text	CHAR(n)	Fixed-length: country codes, flags, short codes	Long, variable strings — wastes space with padding
Date/Time	DATE	Calendar dates with no time component	When you need time-of-day — use TIMESTAMP
Date/Time	TIMESTAMP	Date + time. Default is without time zone	Date-only data — wastes storage vs DATE
Boolean	BOOLEAN	True/false flags: is_active, is_member, has_paid	Multi-state values — use CHAR(1) or SMALLINT
JSON	JSON	String stored JSON – Original format is important	Processing is required in SELECT statements
JSON	JSONB	Binary stored JSON, pre-indexed, fast to process	Formatting of JSON is important

Distribution: How Your Data Is Spread

Distribution is an important performance design decision — it determines how rows are placed across compute nodes and whether joins need expensive network shuffles.

DISTRIBUTE ON (col)

```
DISTRIBUTE ON  
(transaction_id)
```

Best for: Large fact tables. Rows with the same key value land on the same node — enables co-located joins.

When to use: Almost always your choice for big tables. Pick a high-cardinality column that appears in your most common joins.

DISTRIBUTE REPLICATE

```
DISTRIBUTE REPLICATE
```

Best for: Small dimension/lookup tables. A full copy of the table lives on every compute node.

When to use: Tables < a few GB that are joined frequently. Eliminates network shuffle for that join entirely.

⚠ Every DML (INSERT/UPDATE/DELETE) writes to every node — slow for large or frequently-updated tables.

DISTRIBUTE RANDOM

```
DISTRIBUTE RANDOM
```

Best for: Round-robin row assignment. No affinity — rows of the same key can land anywhere.

When to use: Staging / temp tables where you don't care about co-location, or when there's no good distribution key.

⚠ Always causes a redistribution on joins. Only use when HASH or REPLICATE don't fit.

Ordering: Helping the Engine Skip Work

`SORT ON` physically orders data within a shard by the specified columns. Order by will order data bulk loaded into tables. This combination allows block pruning — the engine skips entire blocks that can't match your `WHERE` filter.

Syntax

```
CREATE TABLE mart.casino_revenue (  
  transaction_id BIGINT NOT NULL,  
  property_code VARCHAR(10) NOT NULL,  
  txn_date DATE NOT NULL,  
  revenue_amt NUMERIC(15,2)  
)  
DISTRIBUTE ON (transaction_id)  
SORT ON (txn_date);  
  
INSERT INTO mart.casino_revenue SELECT * FROM  
staging.casino_revenue ORDER BY txn_date;
```

Block Pruning in Action

SKIP Jan 01–Jan 24

READ Mar 12–Apr 04

SKIP Jan 25–Feb 16

SKIP Apr 05–Apr 28

READ Feb 17–Mar 11

SKIP Apr 29–May 31

```
WHERE txn_date BETWEEN  
'2024-02-20' AND '2024-04-01'
```

- Use for a most filtered-on column, sometimes columns are correlated (like `tx_id` and date)
- Need both sort on on the table definition and insert into / order by when bulk loading data

Constraints: What Yellowbrick Enforces (and What It Doesn't)

Yellowbrick is NOT an OLTP database. Most constraints are informational-only — they help the optimizer but do not prevent bad data from being loaded.

Constraint	Enforced?	Example Syntax	What This Means for You
NOT NULL	✓ YES	<code>col VARCHAR(50) NOT NULL</code>	Prevents NULL values from being loaded. Use wherever nulls would break your queries. This is the one constraint you should actively use.
PRIMARY KEY	✗ No	<code>PRIMARY KEY (transaction_id)</code>	Informational only. The optimizer may use it for planning, but duplicate or NULL keys will load without error. Enforce uniqueness in your ETL process.
FOREIGN KEY	✗ No	<code>REFERENCES dim_customer(id)</code>	Informational only. Helps the optimizer understand relationships. Referential integrity must be managed by your data pipeline, not the database.
UNIQUE	✗ No	<code>UNIQUE (email)</code>	Informational only. No index is built. Duplicate values will be accepted. Use for query planning hints only.
CHECK	✗ No	<code>CHECK (amount > 0)</code>	Informational only — conditions are not evaluated on load. Validate data quality in your ETL before inserting.

Temporary Tables & CREATE TABLE AS SELECT

Temporary Tables

Scoped to your session — automatically dropped when you disconnect. Perfect for intermediate results without cluttering shared schemas.

```
-- Create a temp table
CREATE TEMP TABLE temp_active_customers AS
SELECT customer_id, region, spend_ytd
FROM   mart.customers
WHERE  is_active = TRUE;

-- Use it in subsequent queries
SELECT region, AVG(spend_ytd)
FROM   temp_active_customers
GROUP BY region;

-- Dropped automatically on disconnect
-- Or drop manually:
DROP TABLE temp_active_customers;
```

CREATE TABLE AS SELECT (CTAS)

Creates a permanent table from a query result. Inherits column names and types from the SELECT. Distribution defaults to RANDOM unless specified.

```
-- Basic CTAS
CREATE TABLE staging.revenue_2024 AS
SELECT *
FROM   mart.casino_revenue
WHERE  txn_date >= '2024-01-01';

-- CTAS with distribution specified
CREATE TABLE staging.revenue_2024 AS
SELECT *
FROM   mart.casino_revenue
WHERE  txn_date >= '2024-01-01'
ORDER BY txn_date
DISTRIBUTE ON (transaction_id)
SORT ON (txn_date);

ORD
-- ⚠ Always specify DISTRIBUTE ON for
-- large tables — don't rely on RANDOM
```

 **Tip:** Use CTAS to snapshot data before a transformation, or to materialise a complex query for repeated use. Always add DISTRIBUTE ON for tables > a few million rows.

Naming Conventions & Organisation Best Practices

① Use snake_case

Lowercase letters, digits, underscores. No spaces, hyphens, or camelCase. Yellowbrick folds all unquoted names to lowercase.

✓ `casino_revenue_daily` ✗ `Casino Revenue Daily` / `casinoRevenueDaily`

② Be descriptive, not cryptic

Names should be self-explanatory to someone unfamiliar with the data. Avoid abbreviations that only your team understands.

✓ `fact_slot_machine_sessions` ✗ `fct_sms` / `table1`

③ Prefix by table type

Use consistent prefixes to communicate intent: `fact_`, `dim_`, `stg_`, `tmp_`. Makes schemas navigable even with hundreds of tables.

✓ `dim_property` / `fact_revenue` / `stg_raw_bets` ✗ `property` / `revenue` / `raw_bets`

④ Keep personal work in your schema

Create all experimental and personal tables in your own schema (e.g. `msuarez.*`). Keep shared schemas clean and stable.

✓ `msuarez.my_analysis_v3` ✗ `public.test_table_michael`

⑤ Qualify with schema in code

Always write `schema.table` in SQL scripts — never rely on `search_path` to find the right table in production code.

✓ `SELECT * FROM mart.dim_property` ✗ `SELECT * FROM dim_property`

⑥ Avoid reserved words

Don't name columns or tables using SQL keywords (`date`, `table`, `order`, `value`, `user`). If you must, quote them — but then you always have to quote them.

✓ `txn_date` / `record_value` ✗ `"date"` / `"order"`

Common Mistakes to Avoid

X VARCHAR too large

VARCHAR(9999) on every string column wastes storage, hurts compression, and makes the optimizer's estimates wrong.

Fix: Use VARCHAR(50), VARCHAR(200), etc. — size to real-world data, not the maximum imaginable.

X Skewed distribution key

Picking a low-cardinality or uneven column (e.g. gender, status) as the distribution key overloads some nodes and leaves others idle.

Fix: Choose a high-cardinality, evenly spread column — often a transaction ID or customer ID.

! Relying on omitted schema

Writing `SELECT * FROM customers` instead of `mart.customers` works only if `search_path` is set correctly — fragile in scripts.

Fix: Always qualify with `schema.table` in any script or stored procedure that will run in production.

X Missing DISTRIBUTE ON

Creating tables without specifying distribution defaults to RANDOM, causing expensive network shuffles on every join.

Fix: Always set `DISTRIBUTE ON (column)` for fact tables. Think about which column you join on most.

! Creating tables in public

Dropping or altering objects in the public schema affects all users. Personal and experimental tables left there accumulate and confuse everyone.

Fix: Always work in your personal schema or a designated staging schema. Keep public clean.

! Trusting PK/FK constraints

PRIMARY KEY and FOREIGN KEY do not prevent duplicate or orphaned rows. Loading bad data will silently succeed.

Fix: Validate data quality in your ETL pipeline before loading. NOT NULL is the only constraint the database enforces.

Users & Roles

Intro

Users, Roles & How They Connect

Users log in. Roles bundle permissions. GRANTS connect the two.

USER A named login account. You connect to Yellowbrick as a user (with a password).

ROLE A named bundle of permissions. Users can be members of a role and inherit its privileges.

GRANT The SQL command that gives a privilege to a role — or makes a user a member of a role.

PRIVILEGE The actual right: SELECT, INSERT, CONNECT, CREATE, BULK LOAD...

```
CREATE USER msuarez WITH LOGIN PASSWORD '...' ;      CREATE ROLE analyst_role NOLOGIN ;      GRANT  
analyst_role TO msuarez ;
```

Users vs. Roles: The Basics

What is a USER?

A named login account with a password

Connects to Yellowbrick using a client tool (ybsql, DBeaver, ODBC)

Users are instance-wide — not per-database

The same user can access any database they have CONNECT privilege on

All new users belong to the consumer role by default

consumer has no privileges — you start with nothing and access is added via GRANTS

CREATE USER = CREATE ROLE with LOGIN assumed

These are equivalent: CREATE USER alice ... and CREATE ROLE alice LOGIN ...

What is a ROLE?

A named bundle of permissions (NOLOGIN by default)

Roles cannot log in directly — they exist to group privileges together

Users become members of a role via GRANT

GRANT analyst_role TO alice; — now alice inherits all of analyst_role's privileges

Privileges are inherited automatically (INHERIT attribute)

No need to SET ROLE — Alice can just run queries and her permissions apply

Roles can be members of other roles

senior_analyst_role can inherit everything analyst_role has, plus more

Tip: Grant access to roles, not individual users. When Alice moves teams, change her role membership — not 50 individual GRANTS.

Roles as Groups: How Inheritance Works

Grant access to roles — users get it automatically by being members.

Step 1 — Create the Role

Create a role with NOLOGIN (roles don't need passwords):

```
CREATE ROLE
  analyst_role
  NOLOGIN;
```

Now you have an empty role — no privileges yet.

Step 2 — Grant Privileges

Give the role the access you want:

```
GRANT CONNECT
  ON DATABASE melco
  TO analyst_role;
```

```
GRANT USAGE
  ON SCHEMA mart
  TO analyst_role;
```

```
GRANT SELECT ON ALL TABLES
  IN SCHEMA mart
  TO analyst_role;
```

Step 3 — Add Members

Add users (or other roles) to the role:

```
GRANT analyst_role
  TO msuarez;
```

```
GRANT analyst_role
  TO jsmith;
```

```
GRANT analyst_role
  TO rlee;
```

All three users are now members.

Step 4 — Inheritance

Users inherit the role's privileges automatically (INHERIT attribute is the default).

msuarez, jsmith, and rlee can all:

CONNECT to melco database
USAGE on the mart schema
SELECT from mart tables

To revoke access: REVOKE
analyst_role FROM msuarez;

The Permission Hierarchy

Privileges can be granted at 4 levels — from the whole system down to a single table.

ON SYSTEM Instance-wide admin: CREATE ROLE, CREATE DATABASE, TRACE QUERY, CONTROL ANY SESSION

ON DATABASE Per-database: CONNECT (log in), CREATE (schemas), BACKUP, BULK LOAD

ON SCHEMA Per-schema: USAGE (look inside), CREATE (make tables in this schema)

ON TABLE Per-table or per-column: SELECT, INSERT, UPDATE, DELETE, TRUNCATE, REFERENCES

As a business analyst, you typically only need ON DATABASE (CONNECT) + ON SCHEMA (USAGE) + ON TABLE (SELECT). Your DBA handles the rest.

GRANT: Giving Access

Common GRANT Examples

```
-- Add a user to a role
GRANT analyst_role TO msuarez;

-- Allow a role to see inside a schema
GRANT USAGE ON SCHEMA mart
  TO analyst_role;

-- Read-only on all tables in a schema
GRANT SELECT ON ALL TABLES
  IN SCHEMA mart TO analyst_role;

-- Remove a privilege
REVOKE SELECT ON TABLE mart.revenue
  FROM analyst_role;
```

Key Points

Grant to the role, not each user

When a user joins the role they automatically get all its privileges — no individual GRANTS needed.

Privileges are cumulative

New GRANTS add to what already exists. Granting SELECT twice has no extra effect.

Object owners have all privileges by default

The person who created a table already has all rights on it — no GRANT needed.

GRANT ALL doesn't mean superuser

GRANT ALL PRIVILEGES ON TABLE t grants SELECT, INSERT, UPDATE, DELETE, TRUNCATE — not admin powers.

WITH GRANT OPTION lets recipients re-grant

Use sparingly — only give this to trusted roles that need to manage access themselves.

What Analysts Typically Have

A typical business analyst will have a setup like this.

Read: mart & raw	Write: Personal Schema	Not Granted (by default)	How to Request More Access
Via team role (e.g. analyst_role): CONNECT on melco database USAGE on mart schema USAGE on raw schema SELECT on all mart.* tables SELECT on all raw.* tables You can run any SELECT or reporting query against these schemas.	You own your personal schema: <pre>CREATE TABLE msuarez.my_table INSERT / UPDATE / DELETE DROP TABLE msuarez.test</pre> Your personal schema is a safe sandbox — experiment freely without touching shared data.	To protect the shared environment: No CREATEDB Cannot create new databases No BULK LOAD yblast requires explicit DB privilege No BACKUP / RESTORE Admin-only operations No other teams schemas Access is not implied by connection	If you hit a permission denied error: 1. Note the exact object name (schema.tablename or database) 2. Know what you need SELECT only? Or INSERT too? 3. Contact your DBA They apply the GRANT — you don't need to write it yourself.

Checking Your Access

Useful Queries

```
-- Who am I?
SELECT current_user;

-- What roles am I assigned?
SELECT name, memberof
FROM sys.role where name=current_user;

-- Can I SELECT from this table?
SELECT HAS_TABLE_PRIVILEGE(
'mart.casino_revenue',
'SELECT');
```

What These Tell You

current_user

Returns your login name — useful to confirm you connected as the right account.

sys.user

Shows all roles in the instance. Look here to find what roles are available to request.

HAS_TABLE_PRIVILEGE

Returns t (true) or f (false). Also works for HAS_SCHEMA_PRIVILEGE and HAS_DATABASE_PRIVILEGE.

Permission denied is expected

If you get this error — it's not a bug. It means you need to request access. Contact your DBA with the object name and what you need.

Questions?

docs.yellowbrick.com · support.yellowbrick.com · michael.suarez@yellowbrick.com